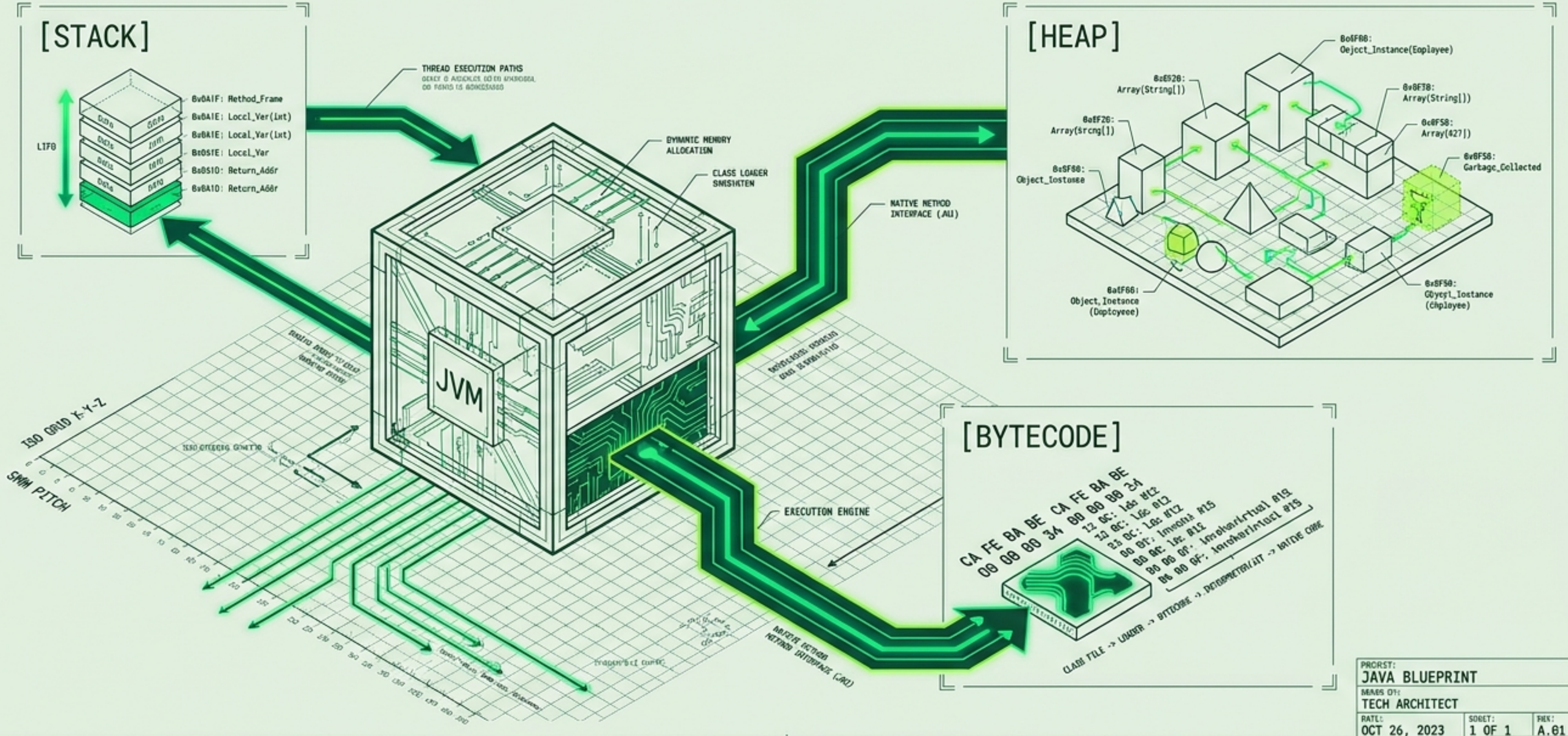


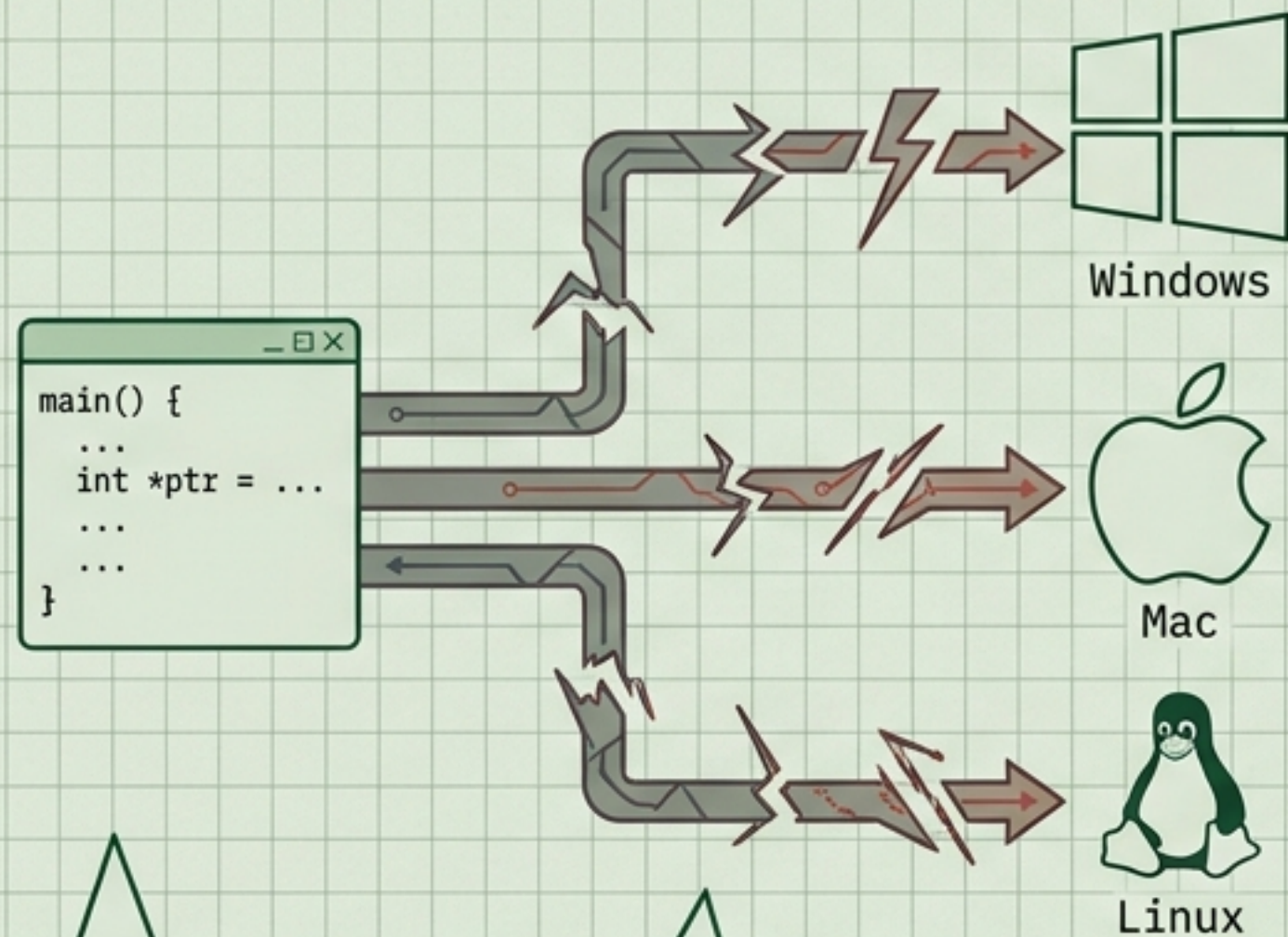
Anatomía de Java: El Blueprint Digital

Fundamentos, Memoria y la Máquina Virtual (Módulo 1)



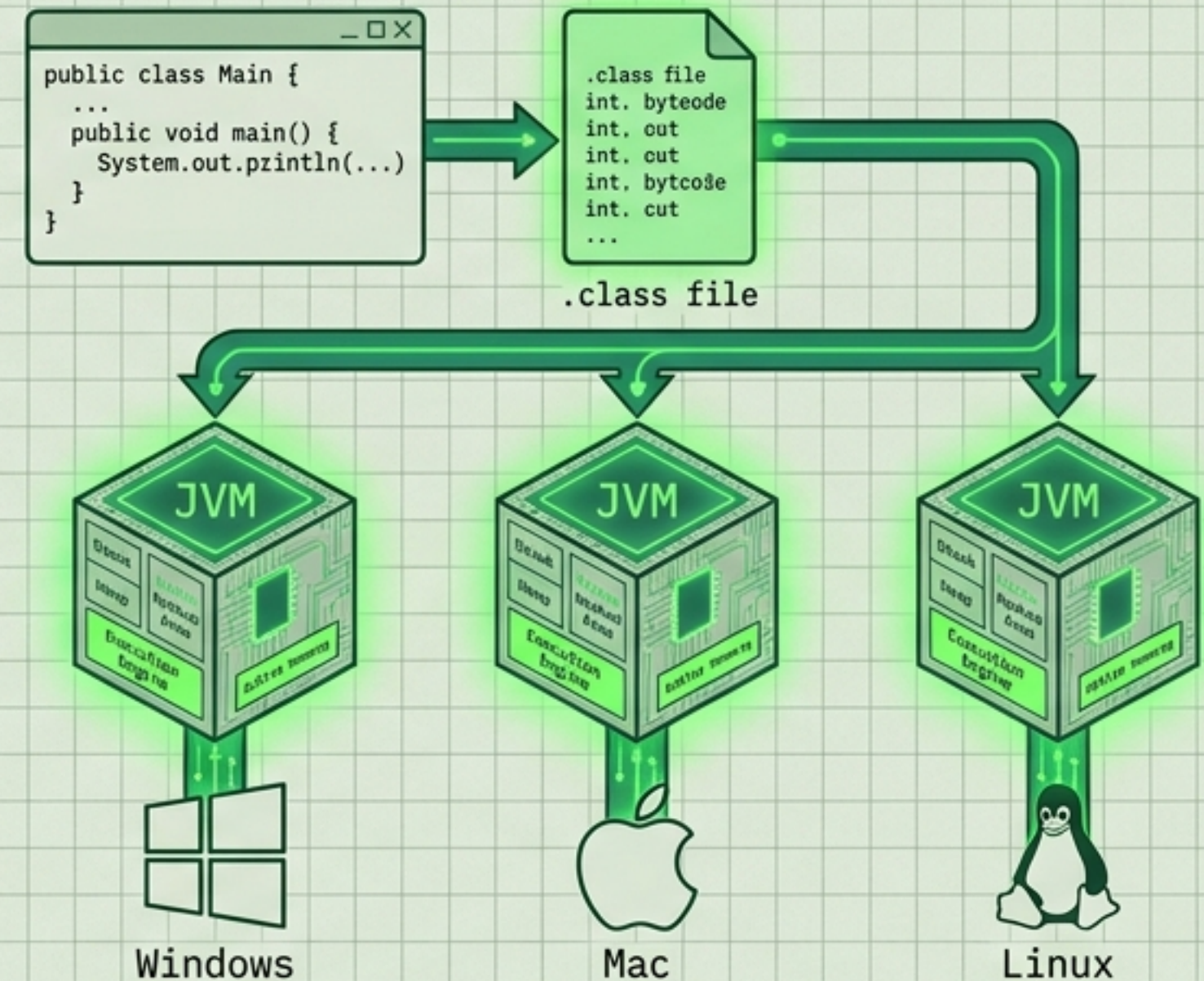
El Origen: La Filosofía de la Máquina Virtual

Pre-1995 (Era C/C++)



Fragmentación y dependencias del sistema operativo.

1995 - Presente (Era Java)



1995: James Gosling y Sun Microsystems crean Java bajo una premisa radical: "Write Once, Run Anywhere".

La Innovación: Introducir una capa de abstracción (La Máquina Virtual) entre el código y el hardware.

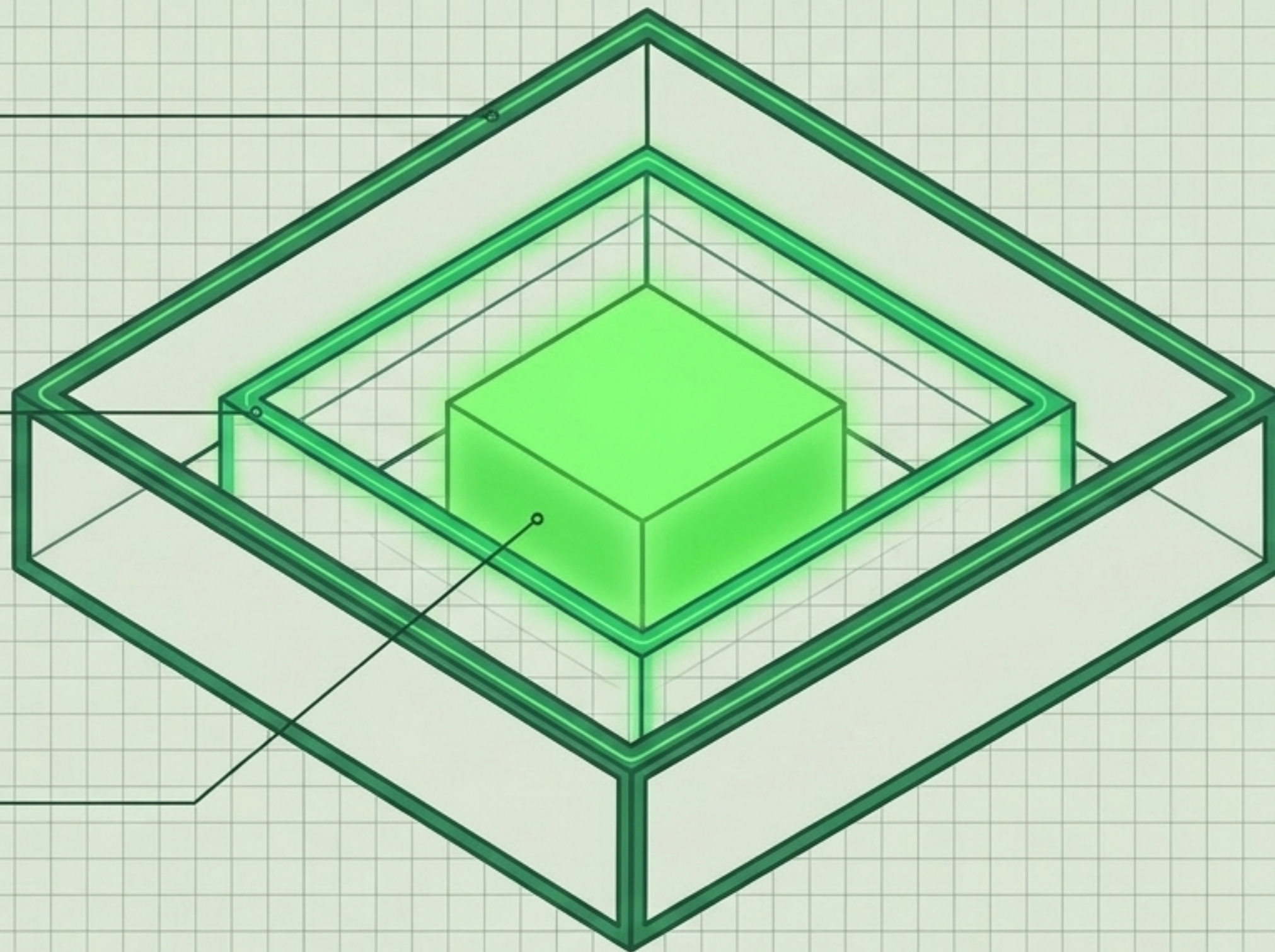
Filosofía de Diseño: Portabilidad absoluta, gestión automática de memoria (Garbage Collector) y tipado estático seguro.

El Ecosistema: Arquitectura Concéntrica

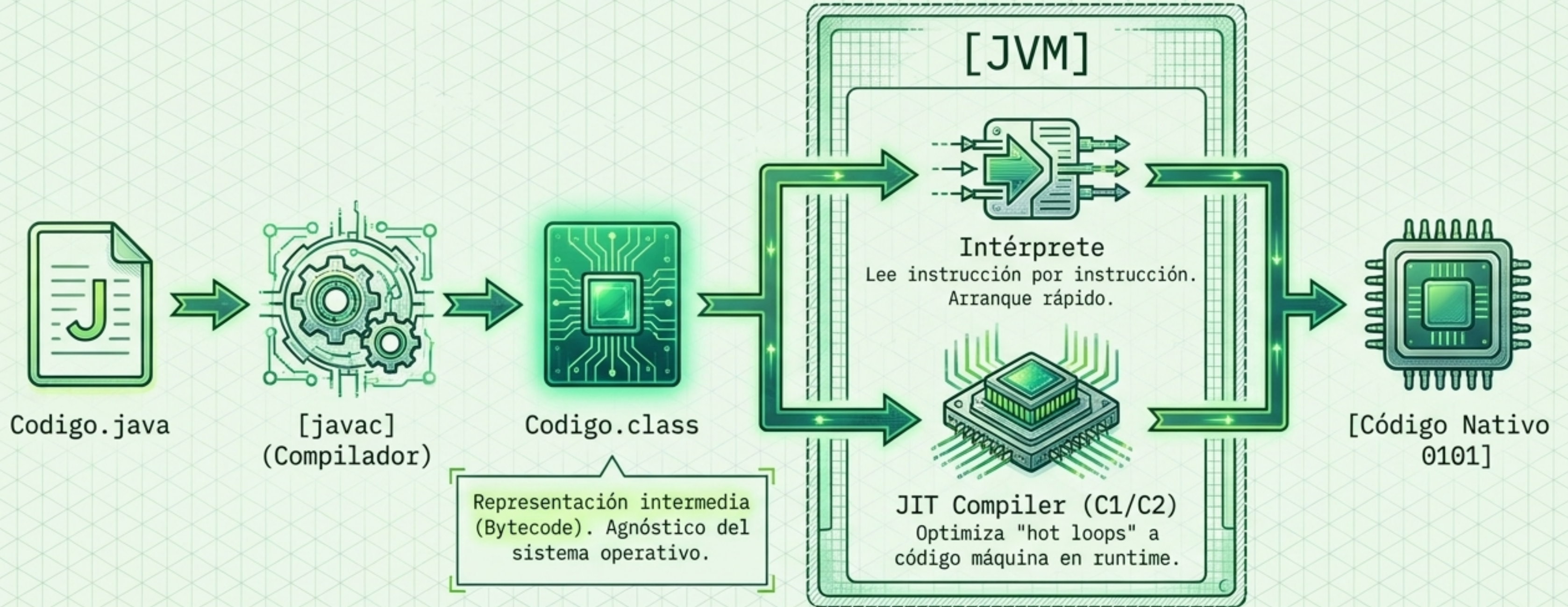
JDK (Java Development Kit)
Para escribir y compilar código.
Contiene compilador (javac) y
herramientas.

JRE (Java Runtime Environment)
El entorno mínimo para ejecutar
una app. Contiene librerías base
(java.lang, java.util).

JVM (Java Virtual Machine)
El motor. Convierte bytecode en
instrucciones nativas del SO.



El Puente al Hardware: Compilación Dual



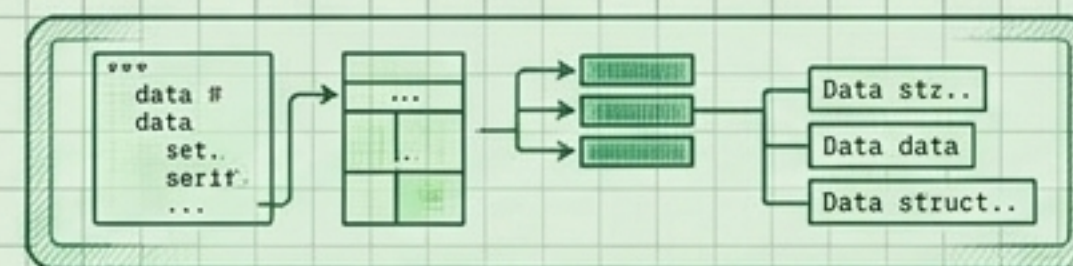
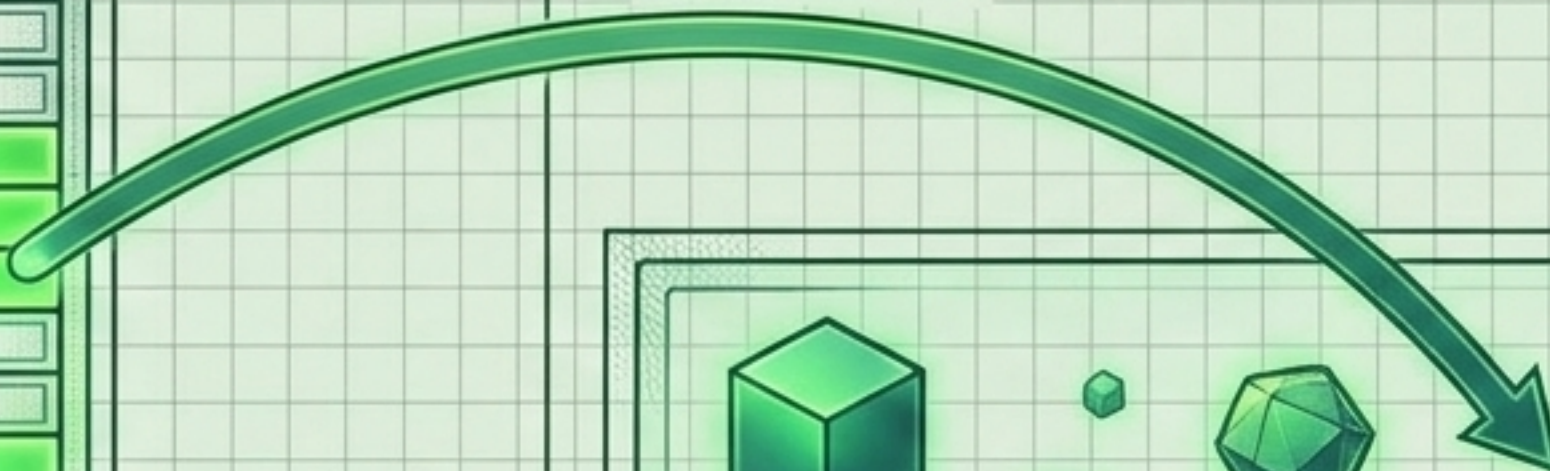
Anatomía de la Memoria: Stack vs Heap



The Stack (Pila)

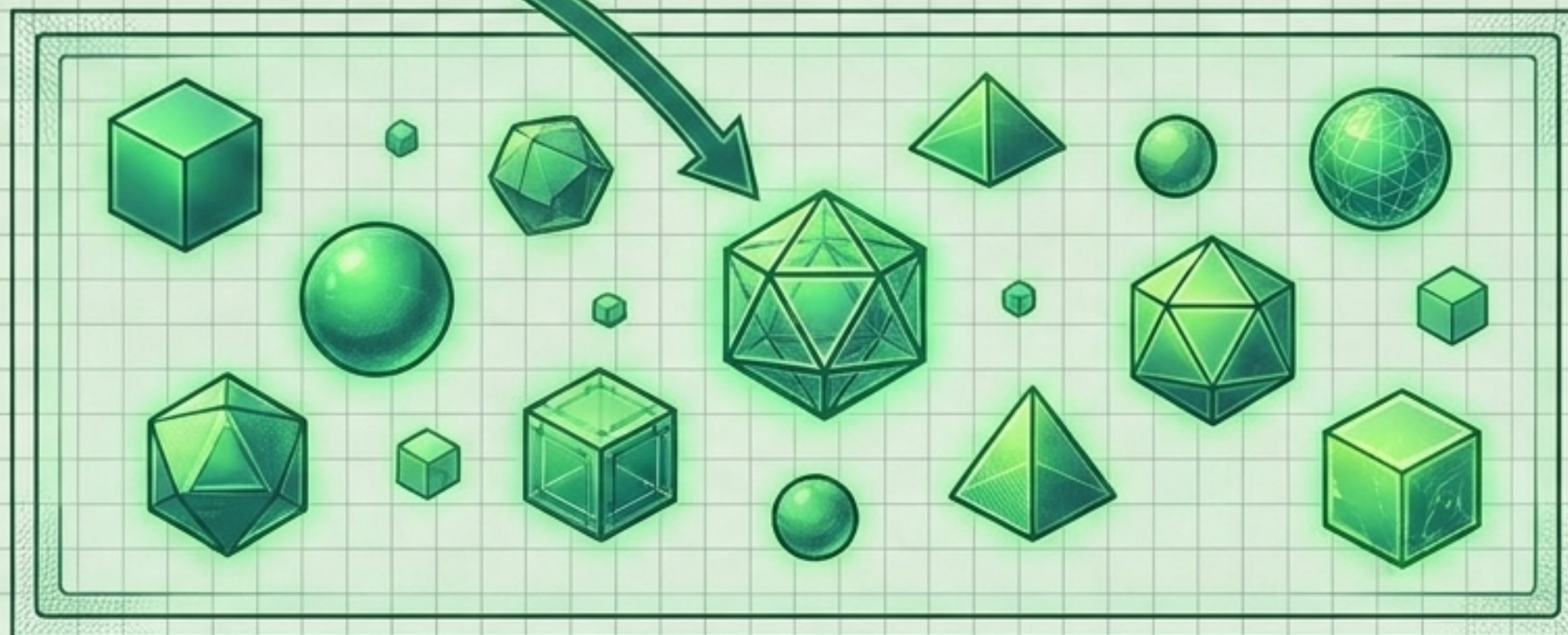
Memoria rápida, local a cada hilo (thread). Almacena variables locales, tipos primitivos y punteros de referencia.

Referencia



Metaspaces

Almacena metadatos de las clases. Crece dinámicamente fuera del Heap.



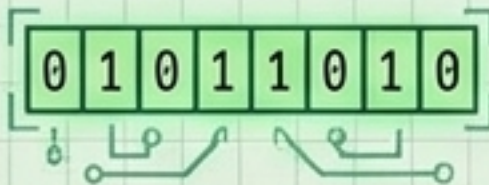
The Heap (Montículo)

Memoria masiva compartida. Todos los objetos y arrays viven aquí. Gestionado automáticamente por el Garbage Collector.

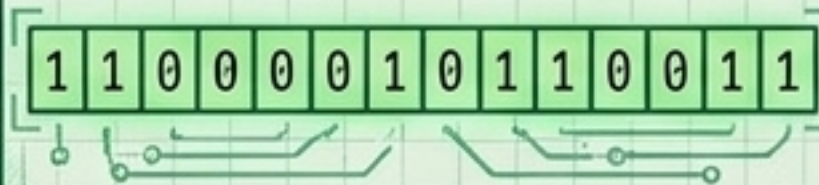
La Materia Prima: 8 Tipos Primitivos

Enteros

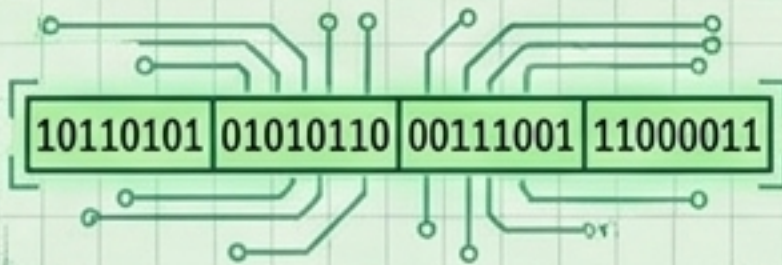
byte (1 byte)



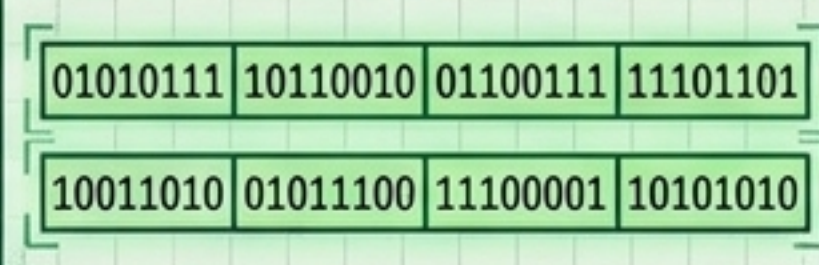
short (2 bytes)



int (4 bytes, default)



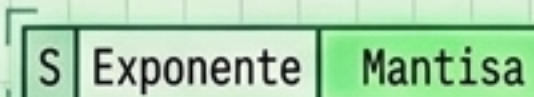
long (8 bytes)



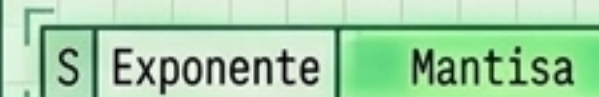
Complemento a dos

Flotantes

float (4 bytes)



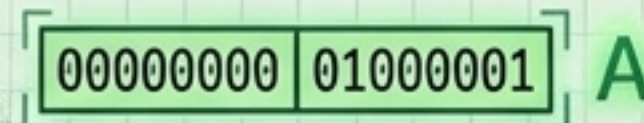
double (8 bytes)



IEEE 754 - Inexactos para divisas

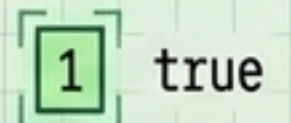
Caracteres

char (2 bytes, Unicode)



Lógicos

boolean (1 bit)

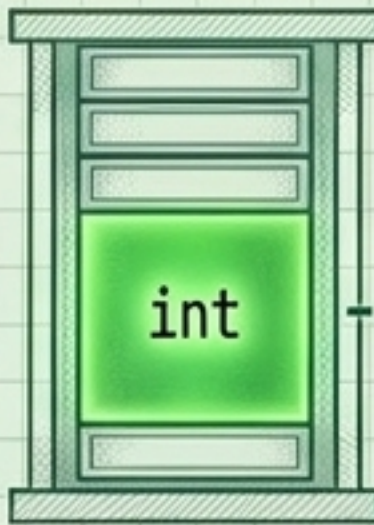


Concepto Clave: Los primitivos **NO son objetos**. No tienen métodos, no tienen cabecera de memoria, y viven directamente en la **pila (Stack)** para garantizar el máximo rendimiento de ejecución.

La Convivencia: Primitivos vs Clases Envolvertes

Tipo Primitivo (ej. int)

The Stack (Pila)

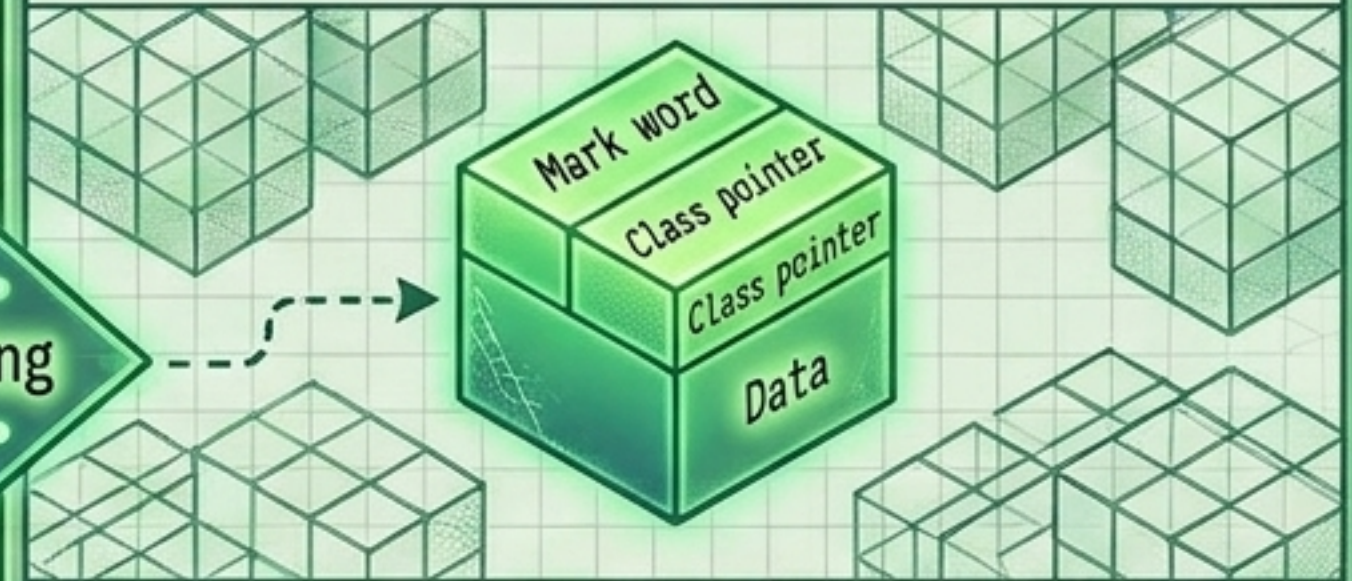


- 4 bytes exactos
- Acceso ultrarrápido
- No puede ser null
- Sin métodos asociados

Autoboxing / Unboxing

Clase Envolverte (ej. Integer)

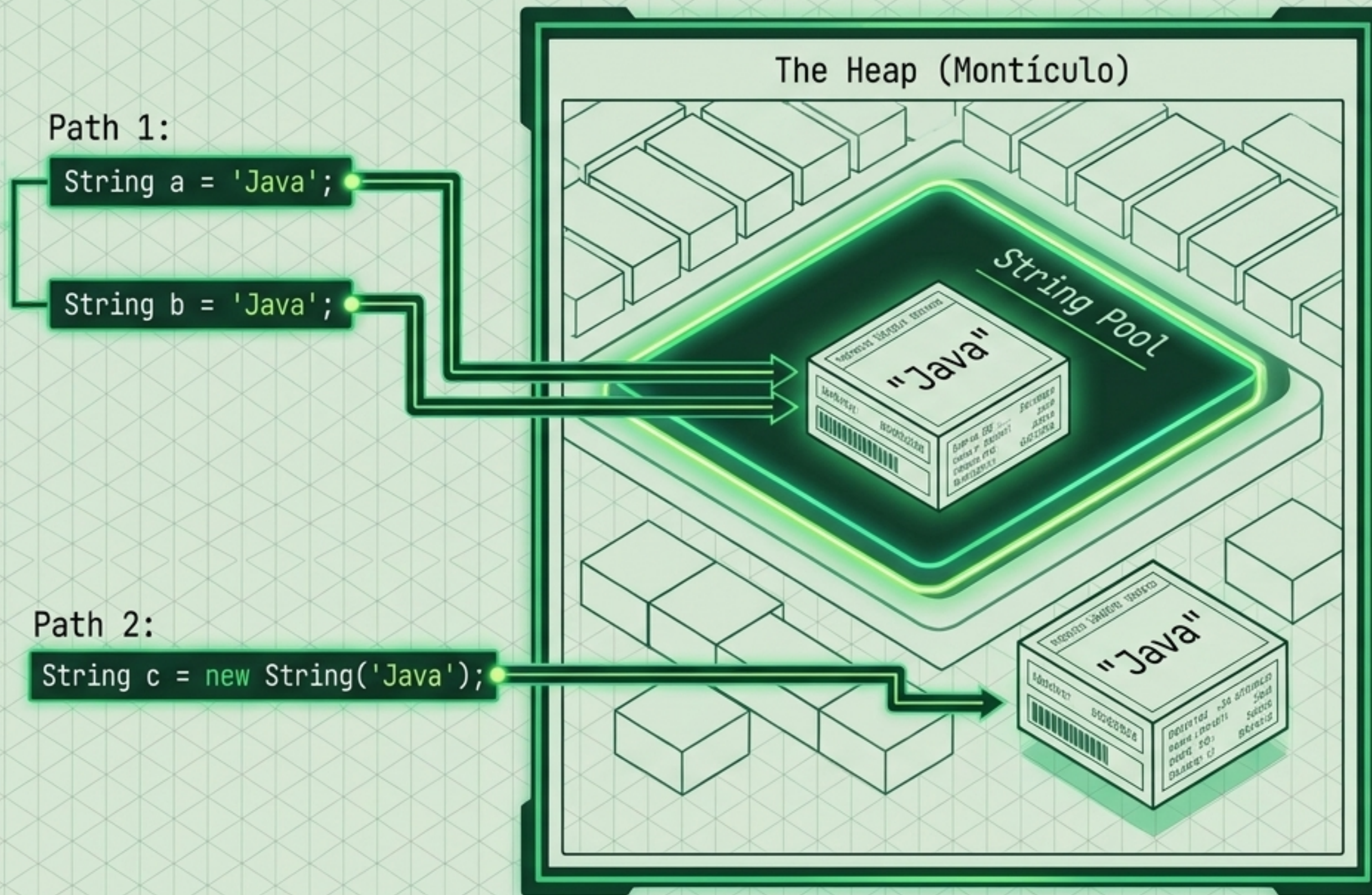
The Heap (Montículo)



- Objeto completo en memoria (pesado)
- Permite valores null
- Provee métodos utilitarios
- Participa en Colecciones (Generics)

El Coste Oculto: Autoboxing empaqueta y desempaqueta valores en runtime. `Integer.valueOf()` cachea valores de -128 a 127. Comparar Integers fuera de este rango con `'=='` es un bug crítico; siempre usa `'equals()'`.

La Peculiaridad del Texto: El String Pool



1. Inmutabilidad:

Por seguridad y concurrencia (hilos), un String nunca cambia.

Modificar un String existente siempre crea un objeto completamente nuevo en memoria.

2. Interning:

La JVM cachea literales de texto en compile-time para ahorrar memoria.

El uso indiscriminado del operador '+' en ciclos inunda el Heap de objetos basura.

Control de Flujo: La Evolución del Switch

Pre-Java 14 (Statement)

```
switch (dia) {  
  case LUNES:  
    tipo = 1;  
    break;  
  // verboso, riesgo de fall-through  
}
```

Java 14+ (Expression)

```
var tipo = switch (dia) {  
  case LUNES -> 1;  
  case MARTES -> 2;  
};
```

Asignación directa, sintaxis limpia, sin necesidad de break.

tableswitch

Dense cases, $O(1)$
jump array

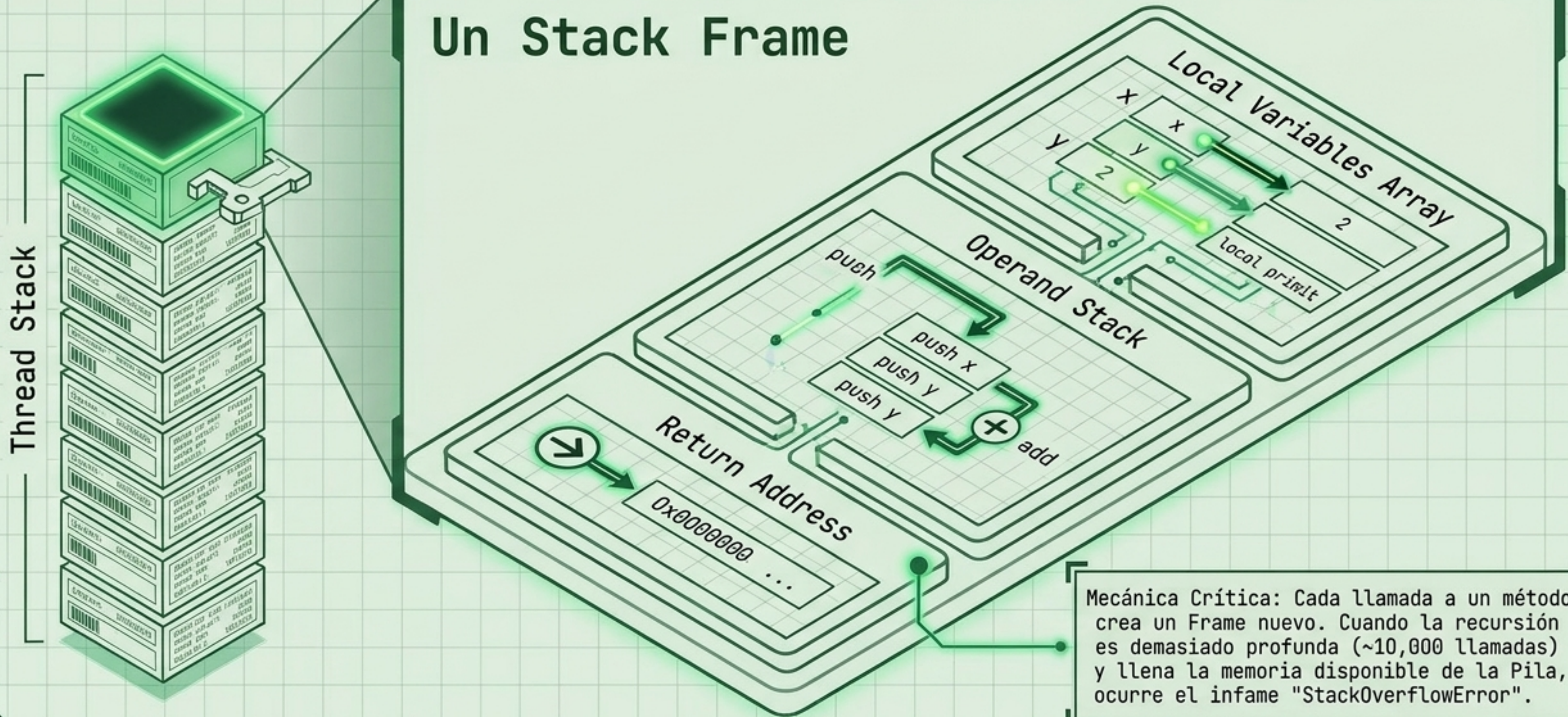
JIT Compiler
(Procesamiento)

lookupswitch

Sparse cases,
Binary search tree

Anatomía de la Ejecución: El Stack Frame

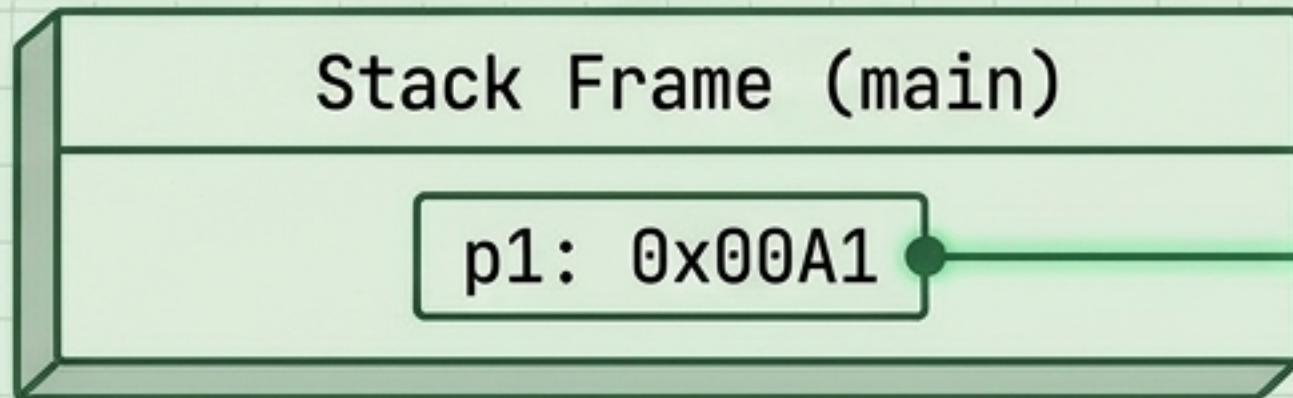
Un Stack Frame



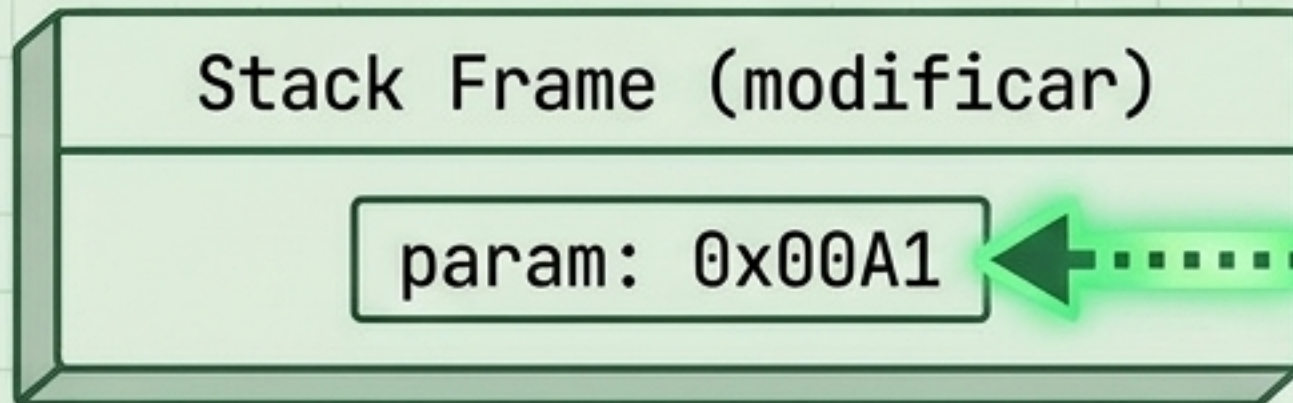
Mecánica Crítica: Cada llamada a un método crea un Frame nuevo. Cuando la recursión es demasiado profunda (~10,000 llamadas) y llena la memoria disponible de la Pila, ocurre el infame "StackOverflowError".

La Gran Regla: Paso por Valor (Siempre)

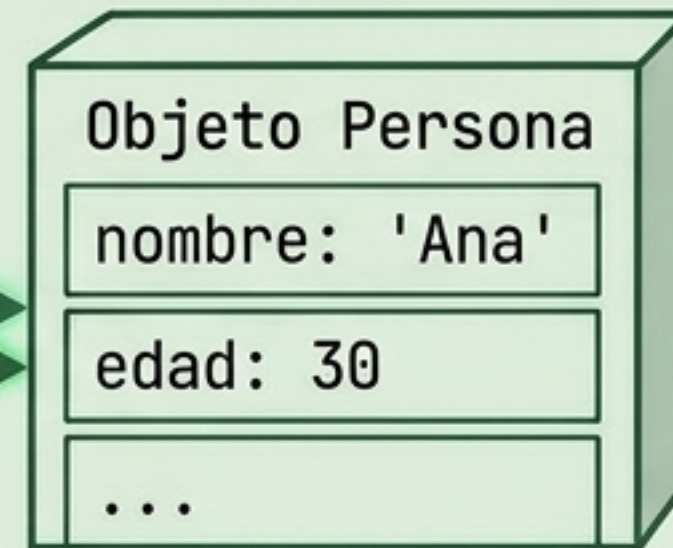
Estado 1: Caller



Estado 2: Method Call



Heap

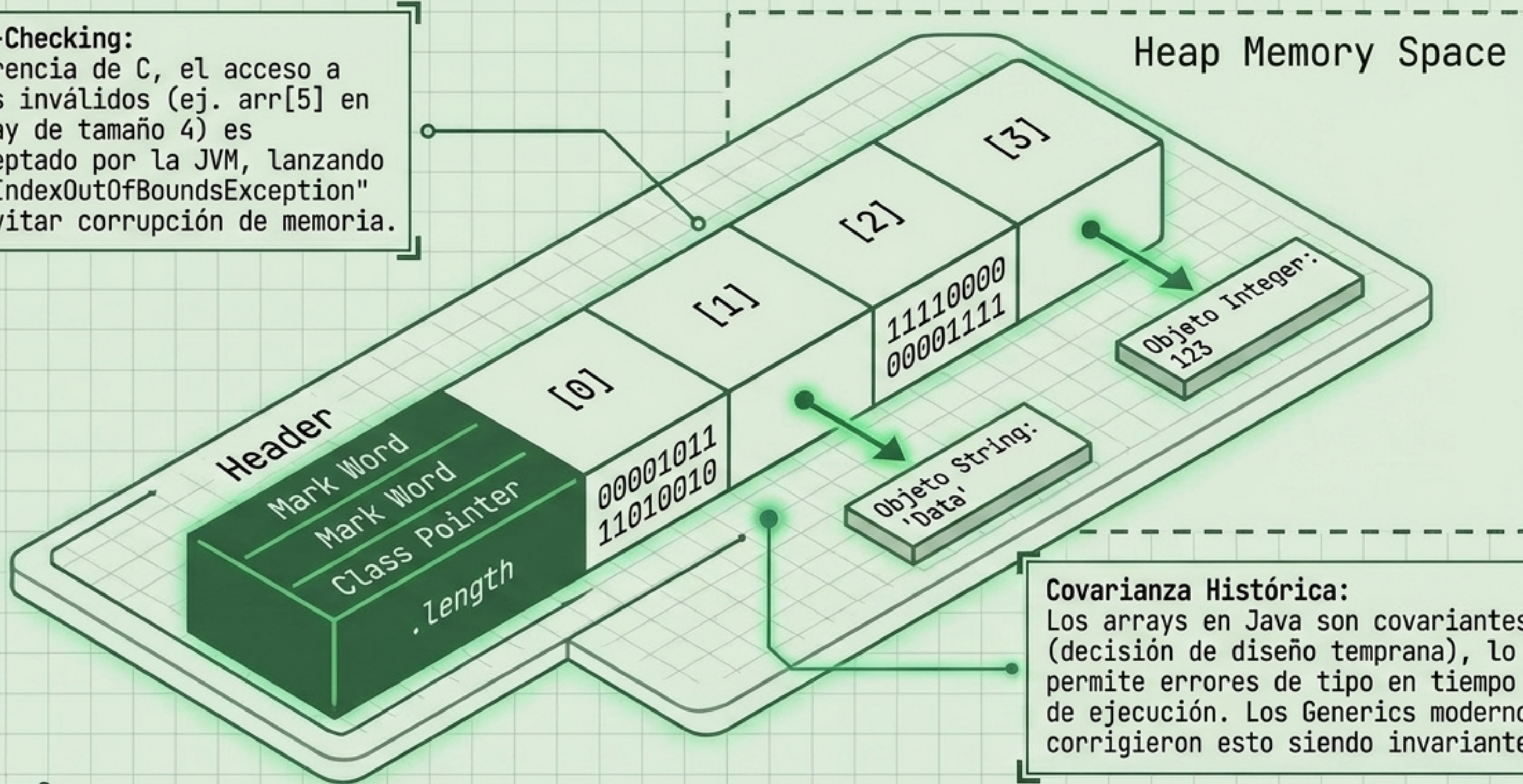


Java NUNCA pasa objetos por referencia. Pasa el *valor de la referencia* por valor. Puedes mutar el objeto interno, pero no puedes reasignar la variable original. A este comportamiento se le denomina *Call-by-sharing*.

Colecciones Base: Arrays como Objetos

Bounds-Checking:

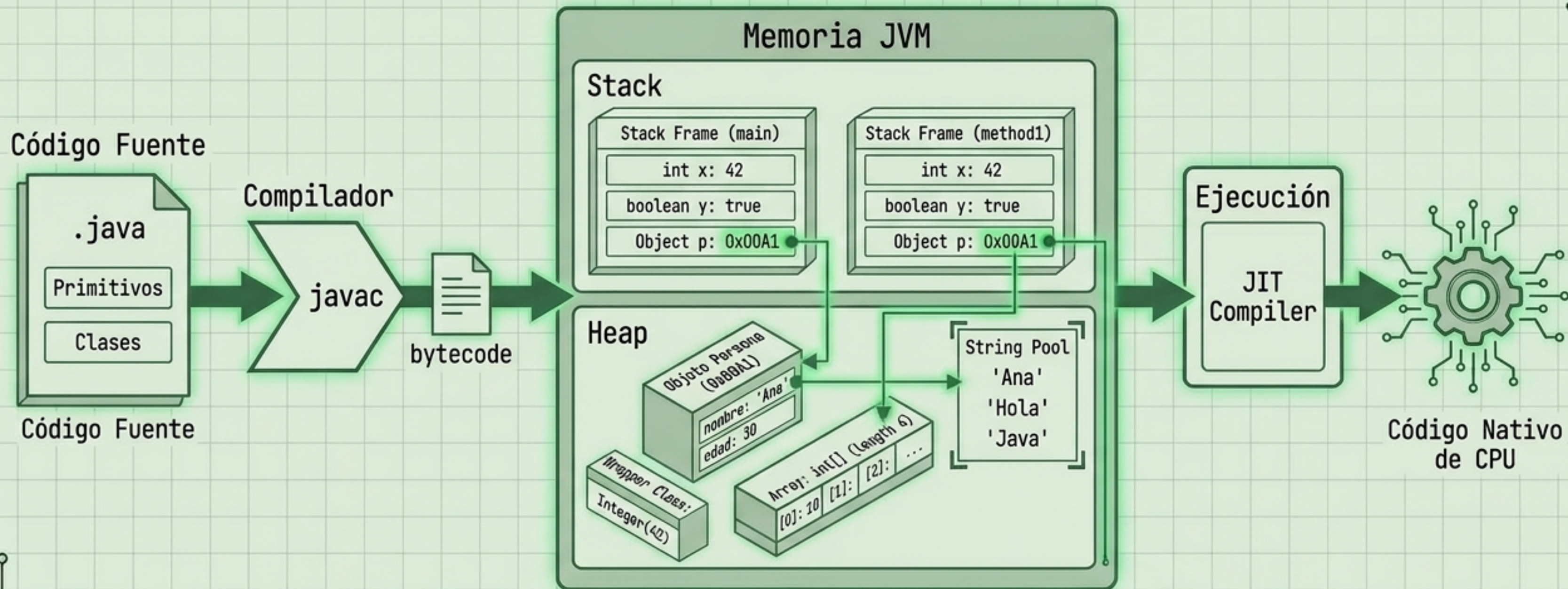
A diferencia de C, el acceso a índices inválidos (ej. `arr[5]` en un array de tamaño 4) es interceptado por la JVM, lanzando `"ArrayIndexOutOfBoundsException"` para evitar corrupción de memoria.



Covarianza Histórica:

Los arrays en Java son covariantes (decisión de diseño temprana), lo que permite errores de tipo en tiempo de ejecución. Los Generics modernos corrigieron esto siendo invariantes.

Síntesis: El Ciclo de Vida en la Máquina



Java es un ecosistema de ingeniería diseñado para abstraer la complejidad del hardware sin perder el rigor sobre el tipado, la memoria y la estructura arquitectónica de los datos.